Nama : Aura Rieska Maharani

NIM : 434241051

Kelas : C4

Github : https://github.com/aurarieska/434241051_BackendPrak

# LAPORAN PRAKTIKUM MODUL 2 BACKEDN LANJUT

## Base Pemahaman dan Panduan

1. **Anatomi HTTP**
2. **Metode HTTP : Safe dan Idempotent**

   Safe berarti tidak mengubah apa pun di server. Aman diulang, aman di-cache, aman di-crawl mesin pencari.

   Idempotent berarti mengirim permintaan yang sama sekali atau seratus kali memberi hasil akhir yang sama. Penting sekali ketika jaringan buruk dan klien mengulang permintaan.

| Metode | Safe | Idempotent | Dipakai untuk |
|---|---|---|---|
| GET | Ya | Ya | Mengambil data |
| HEAD | Ya | Ya | Sama seperti GET, tetapi hanya header tanpa body |
| OPTIONS | Ya | Ya | Menanyakan kemampuan endpoint |
| POST | Tidak | Tidak | Membuat sumber daya baru |
| PUT | Tidak | Ya | Mengganti seluruh isi sumber daya |
| PATCH | Tidak | Tidak selalu | Mengubah sebagian isi sumber daya |
| DELETE | Tidak | Ya | Menghapus sumber daya |

3. **Status HTTP**

| Kelompok | Arti umum |
|---|---|
| 2xx | Berhasil |
| 3xx | Perlu langkah tambahan, biasanya pengalihan |
| 4xx | Kesalahan ada di pihak klien |
| 5xx | Kesalahan ada di pihak server |

4. **Main**

```
var metodeBerbody = map[string]bool{
    fiber.MethodPost:  true,
    fiber.MethodPut:   true,
    fiber.MethodPatch: true,
}
```

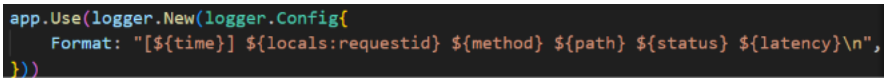
metodeBerbody digunakan untuk menyimpan daftar HTTP method yang dianggap memiliki request body, yaitu POST, PUT, dan PATCH. Data disimpan dalam bentuk map dengan key berupa nama method dan value berupa bool. Nilai true berarti method tersebut termasuk method yang akan diperiksa oleh middleware requireJSON. Jadi ketika nanti c.Method() menghasilkan POST, kode dapat mengecek metodeBerbody["POST"] dan mendapatkan true.

```
// requireJSON menolak request berisi body yang Content-Type-nya bukan JSON.
// Status yang tepat untuk kasus ini adalah 415, bukan 400.
func requireJSON(c *fiber.Ctx) error {
    if metodeBerbody[c.Method()] {
        ct := c.Get("Content-Type")
        if !strings.HasPrefix(ct, fiber.MIMEApplicationJSON) {
            return fail(c, fiber.StatusUnsupportedMediaType,
                "Content-Type harus application/json")
        }
    }
    return c.Next()
}
```

Function ini digunakan sebagai middleware untuk memastikan request POST, PUT, dan PATCH menggunakan Content-Type: application/json. Function menerima c bertipe *fiber.Ctx yang diberikan oleh Fiber ketika request masuk. c.Method() digunakan untuk mendapatkan HTTP method dari request, kemudian hasilnya digunakan sebagai key untuk mengecek metodeBerbody. Jika method tersebut termasuk POST, PUT, atau PATCH, function mengambil header Content-Type dari request menggunakan c.Get("Content-Type"). Jika header tersebut bukan JSON, function menghentikan request dan mengembalikan status 415 Unsupported Media Type. Jika method tidak membutuhkan body atau Content-Type sudah benar, function menjalankan c.Next() untuk meneruskan request ke handler berikutnya.

```
// Middleware global — urutan pemasangan menentukan urutan eksekusi
app.Use(requestid.New())
```

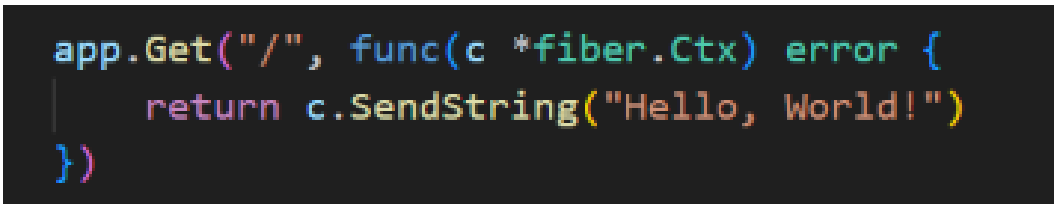
Kode ini digunakan untuk memasang middleware requestid secara **global** pada aplikasi. Middleware ini memberikan ID unik pada setiap request yang masuk. Karena dipasang menggunakan app.Use(), middleware ini berlaku untuk seluruh endpoint yang ada di aplikasi.



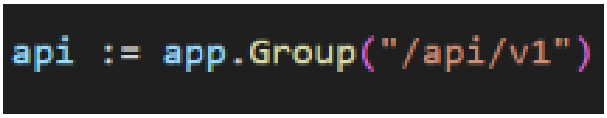

Kode ini digunakan untuk memasang middleware logger secara **global** agar setiap request dicatat ke terminal. Logger mengambil informasi dari request dan proses server seperti waktu request, request ID, HTTP method, path, status response, dan waktu yang dibutuhkan request. Nilai-nilai tersebut berasal dari informasi yang tersedia di Fiber selama request diproses.



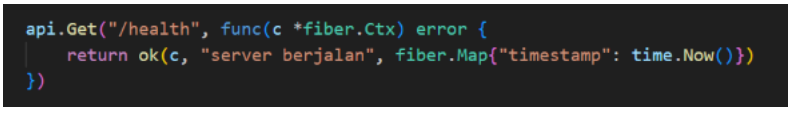

Kode ini digunakan untuk memasang middleware CORS secara **global**. Middleware ini menangani aturan Cross-Origin Resource Sharing sehingga request dari origin yang berbeda dapat diproses sesuai konfigurasi CORS aplikasi.




Kode ini digunakan untuk membuat endpoint GET /. Function menerima c yang diberikan oleh Fiber ketika endpoint dipanggil. Function kemudian menggunakan c.SendString() untuk mengirim teks "Hello, World!" sebagai response. Jadi ketika client mengakses GET http://localhost:3000/, server mengembalikan teks tersebut.




Kode ini digunakan untuk membuat route group dengan prefix /api/v1 dan menyimpannya ke variabel api. Setelah group dibuat, setiap endpoint yang didaftarkan menggunakan api otomatis memiliki prefix /api/v1. Misalnya api.Get("/health", ...) menghasilkan endpoint GET /api/v1/health.




Kode ini digunakan untuk membuat endpoint GET /api/v1/health yang berfungsi untuk mengecek apakah server sedang berjalan. Function menerima c dari Fiber dan tidak mengambil data dari URL atau JSON body. Response berisi pesan "server berjalan" dan timestamp yang nilainya dibuat langsung oleh program menggunakan time.Now(). Response kemudian dikirim menggunakan function ok().

```
// requireJSON dipasang khusus pada grup ini, bukan global
u := api.Group("/users", requireJSON)
u.Get("/", listUsers)
u.Get("/:id", getUser)
u.Post("/", createUser)
u.Put("/:id", replaceUser)
u.Patch("/:id", patchUser)
u.Delete("/:id", deleteUser)
```

Kode ini digunakan untuk membuat route group /users di dalam group /api/v1 dan memasang middleware requireJSON khusus untuk group tersebut. Karena api memiliki prefix /api/v1 dan u memiliki prefix /users, maka endpoint yang menggunakan u akan memiliki prefix /api/v1/users. Middleware requireJSON akan dijalankan pada request yang masuk ke endpoint dalam group u, sehingga pengecekan Content-Type hanya berlaku untuk route users dan tidak berlaku secara global.

**u.Get("/", listUsers)**

Kode ini digunakan untuk membuat route group /users di dalam group /api/v1 dan memasang middleware requireJSON khusus untuk group tersebut. Karena api memiliki prefix /api/v1 dan u memiliki prefix /users, maka endpoint yang menggunakan u akan memiliki prefix /api/v1/users. Middleware requireJSON akan dijalankan pada request yang masuk ke endpoint dalam group u, sehingga pengecekan Content-Type hanya berlaku untuk route users dan tidak berlaku secara global.

**u.Get("/:id", getUser)**

Kode ini digunakan untuk membuat endpoint GET /api/v1/users/:id yang digunakan untuk mengambil satu user berdasarkan ID. Nilai :id berasal dari URL dan nantinya dapat diambil oleh getUser menggunakan c.Params("id"). Contohnya ketika client mengakses /api/v1/users/10, maka c.Params("id") menghasilkan "10" dan nilai tersebut kemudian diproses oleh paramID().

**u.Post("/", createUser)**

Kode ini digunakan untuk membuat endpoint POST /api/v1/users/ dan menggunakan createUser sebagai handler. Ketika request masuk, request terlebih dahulu melewati middleware requireJSON karena middleware tersebut dipasang pada group u. Jika Content-Type valid, request diteruskan ke createUser, yang kemudian mengambil data user dari JSON request body menggunakan c.BodyParser(&req)

**u.Put("/:id", replaceUser)**

Kode ini digunakan untuk membuat endpoint PUT /api/v1/users/:id dan menggunakan replaceUser sebagai handler. ID user berasal dari URL melalui c.Params("id"), sedangkan data yang digunakan untuk mengganti user berasal dari JSON request body

melalui c.BodyParser(&req). Sebelum sampai ke replaceUser, request juga melewati middleware requireJSON.

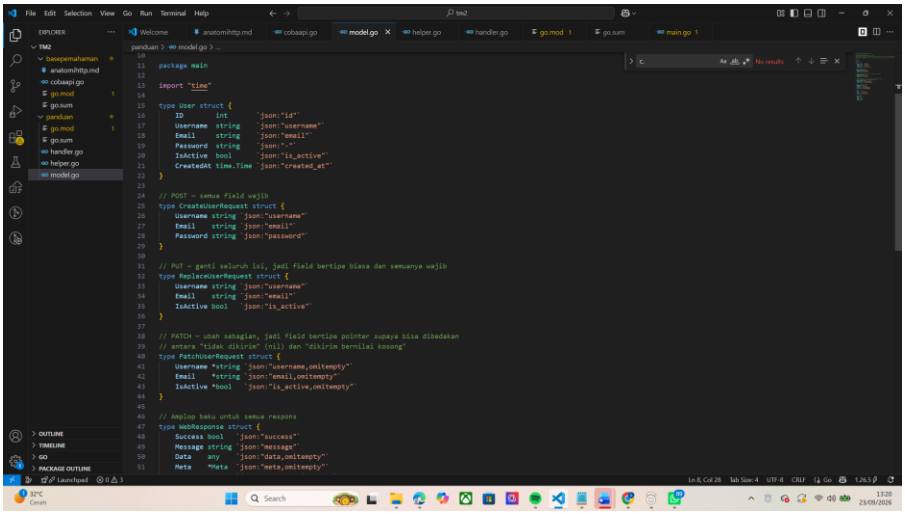
**u.Patch("/:id", patchUser)**

Kode ini digunakan untuk membuat endpoint PATCH /api/v1/users/:id dan menggunakan patchUser sebagai handler. ID user diperoleh dari URL, sedangkan field yang ingin diubah diperoleh dari JSON request body. Request terlebih dahulu diperiksa oleh requireJSON, kemudian diteruskan ke patchUser jika Content-Type sesuai.

**u.Delete("/:id", deleteUser)**

Kode ini digunakan untuk membuat endpoint DELETE /api/v1/users/:id dan menggunakan deleteUser sebagai handler. ID user berasal dari URL dan diambil melalui c.Params("id"). Karena DELETE tidak termasuk dalam metodeBerbody, middleware requireJSON tidak mewajibkan Content-Type: application/json untuk request ini. Setelah itu deleteUser mencari user berdasarkan ID dan menghapusnya.

## 5. Model



Model dipisah ke berbagai struct sesuai kebutuhan field yang diakses. Seperti User dengan CreateUserRequest. Sebenernya bisa aja kalua digabung, untuk semua yang melibatkan user pakai struct User, tetapi gak aman karena ada celah untuk akses field lain (misal harusnya CreateUserRequest itu post hanya untuk field tertentu malah bisa jadi celah untuk post atau menimpa semua field)

## 6. Helper

Info penting : (supaya paham konteks)

fiber.StatusOK // 200
fiber.StatusCreated // 201
fiber.StatusNoContent // 204
fiber.StatusBadRequest // 400
fiber.StatusUnauthorized // 401
fiber.StatusForbidden // 403
fiber.StatusNotFound // 404
fiber.StatusUnprocessableEntity // 422
fiber.StatusInternalServerError // 500

Kumpulan function untuk mempermudah, karena dipanggil berkali-kali supaya tidak rewrite code dari awal.

```go
func ok(c *fiber.Ctx, message string, data any) error {
    return c.Status(fiber.StatusOK).JSON(WebResponse{
        Success: true, Message: message, Data: data,
    })

    // penggunaan : ok(c, "isiMessage", namaVar)
}
```

Function ini kerjanya untuk mengatur status HTTP = 200 (berhasil) response dengan c.Status kemduian memberi respon JSON dengan pesan dan data yang dimasukkan melalui penyebutan namaVar dalam parameter.

```go
func okList(c *fiber.Ctx, message string, data any, meta *Meta) error {
    return c.Status(fiber.StatusOK).JSON(WebResponse{
        Success: true, Message: message, Data: data, Meta: meta,
    })
    // penggunaan oklist(c, "isiMessage", namaVarData, namaVarMeta)
}
```

Mirip dengan function ok tapi terdapat tambahan informasi meta (aturan page, limit, total, dan totalpage)

```go
func created(c *fiber.Ctx, message string, data any, location string) error {
    c.Set("Location", location) // memberi tahu klien di mana sumber daya baru berada
    return c.Status(fiber.StatusCreated).JSON(WebResponse{
        Success: true, Message: message, Data: data,
    })
}
```

Mirip ok tapi statusnya 201 karena pakai c.Status dengan parameter fiber.StatusCreated

```go
func noContent(c *fiber.Ctx) error {
    return c.SendStatus(fiber.StatusNoContent) // 204: berhasil, tanpa body
    // penggunaan noContent(c)
}
```

Function untuk set status http 204, berhasil tanpa body dan gak dikasih pesan json apapun karena di functionnya gak pake apa apa, klo mau ditambahin ya bisa aja

```go
func fail(c *fiber.Ctx, status int, message string) error {
    return c.Status(status).JSON(WebResponse{Success: false, Message: message})
}
```

Untuk menampilkan message gagal

```go
func failValidation(c *fiber.Ctx, errs map[string]string) error {
    return c.Status(fiber.StatusUnprocessableEntity).JSON(WebResponse{
        Success: false, Message: "validasi gagal", Errors: errs,
    })
    // buat pesan validasi
    // penggunaan failValidation("namaField" : "isiMessage", dst)
}
```

Atur status http gagal dan menampilkan pesan detail gagalnya, maka dari itu parameter menggunakan map.

Contoh
failValidation(“username” : “terlalu pendek, minimal 100 karakter”, “email” : “sudah ada”) → atau sebutin variabel mapnya, karena isinya beda-beda, ada case untuk abcd.

```go
// Daftar putih field yang boleh dipakai untuk mengurutkan.
var allowedSort = map[string]bool{
    "id": true, "username": true, "email": true, "created_at": true,
}
```

Untuk menampilkan list field yang diperbolehkan untuk sorting. Digunakan untuk menyusun function penerapan batas list.

```go
// parseListQuery membaca query string dan memberi nilai bawaan yang aman.
// Aturan pentingnya: masukan dari klien tidak pernah dipercaya begitu saja.
func parseListQuery(c *fiber.Ctx) ListQuery {
    q := ListQuery{
        Page:   c.QueryInt("page", 1),
        Limit:  c.QueryInt("limit", 10),
        Search: strings.TrimSpace(c.Query("search")),
        Sort:   c.Query("sort", "id"),
        Order:  strings.ToLower(c.Query("order", "asc")),
    }

    if q.Page < 1 {
        q.Page = 1
    }
    if q.Limit < 1 {
        q.Limit = 10
    }
    if q.Limit > 100 { // batas atas wajib ada
        q.Limit = 100
    }
    if !allowedSort[q.Sort] { // daftar putih, bukan daftar hitam
        q.Sort = "id"
    }
    if q.Order != "desc" {
        q.Order = "asc"
    }

    if raw := c.Query("is_active"); raw != "" {
        if v, err := strconv.ParseBool(raw); err == nil {
            q.IsActive = &v
        }
    }

    return q
}
```

Mengatur kriteria dan batasan penerapan list. Yang diperiksa adalah urlnya GET /users?page=2&limit=150&sort=username&order=desc.

7. **Handler**

| | | | |
|---|---|---|---|
| Dari link url | | | |
| /users?page=2&limit=20&search=aura&sort=username&order=desc | c.QueryInt() | Page: c.QueryInt("page", 1) | q.Page = 2 |
| /users?page=2&limit=20&search=aura&sort=username&order=desc&is_active=true | c.Query() | Sort: c.Query("sort", "id") | q.Search = "aura" |
| /users/10 | c.Params() | c.Params("id") | "10" |
| 2. JSON Body | | | |
| {"username":"aura","email":"aura@gmail.com","password":"12345678"} | c.BodyParser() | c.BodyParser(&req)<br><br>req adalah nama variabel dari tipe data struct | req.Username = "aura" |
| 3. Data Tersimpan | | | |
| users = []User{{ID: 1, Username: "budi", ...}} | - | user[i].ID<br><br>user adalah namaVar | mengambil ID dari data users |

```go
// Bagian 1: penyimpanan dan bantuan
var users []User
var nextID = 1

func findUserIndex(id int) int {
    for i := range users {
        if users[i].ID == id {
            return i
        }
    }
    return -1 //eror untuk data tidak ditemukan
}
```

Function ini digunakan untuk mencari index user berdasarkan ID di dalam slice users. Function menerima id bertipe int. Pada alur kode getUser() dan function lain yang menggunakannya, nilai id tersebut sudah diperoleh sebelumnya dari URL melalui paramID(). Function kemudian mengecek setiap user dalam slice users dengan membandingkan users[i].ID, yaitu ID yang berasal dari data user yang tersimpan,

dengan id, yaitu ID yang berasal dari input URL. Jika ID ditemukan, function mengembalikan index user tersebut. Jika tidak ditemukan, function mengembalikan -1.

```go
// cocokPencarian memeriksa apakah kata kunci muncul di username atau email.
func cocokPencarian(u User, kata string) bool {
    kata = strings.ToLower(kata)
    return strings.Contains(strings.ToLower(u.Username), kata) ||
        strings.Contains(strings.ToLower(u.Email), kata)
}
```

Function ini digunakan untuk mencocokkan kata pencarian dengan username atau email user. Function menerima u bertipe User yang berasal dari data users yang sedang diperiksa, sedangkan kata bertipe string berasal dari q.Search. q.Search sebelumnya diambil dari query URL melalui c.Query("search"). Function kemudian mengubah kata, username, dan email menjadi huruf kecil agar pencarian tidak membedakan huruf besar dan kecil. Hasilnya true jika kata ditemukan pada username atau email, dan false jika tidak ditemukan.

```go
func paramID(c *fiber.Ctx) (int, bool) {
    id, err := strconv.Atoi(c.Params("id"))
    if err != nil || id < 1 {
        return 0, false
    }
    return id, true
}
```

Function ini digunakan untuk mengambil dan memvalidasi ID dari URL path. Function menerima c bertipe *fiber.Ctx yang diberikan oleh Fiber ketika endpoint menerima request. c.Params("id") mengambil nilai id dari URL path, misalnya /users/10. Nilai tersebut awalnya berupa string "10", kemudian strconv.Atoi() mengubahnya menjadi integer 10. Jika nilai bukan angka atau kurang dari 1, function mengembalikan 0 dan false. Jika valid, function mengembalikan ID dan true.

```go
// bagian 2 : daftar dengan saring, urut, dan penggal
func listUsers(c *fiber.Ctx) error {
    q := parseListQuery(c)

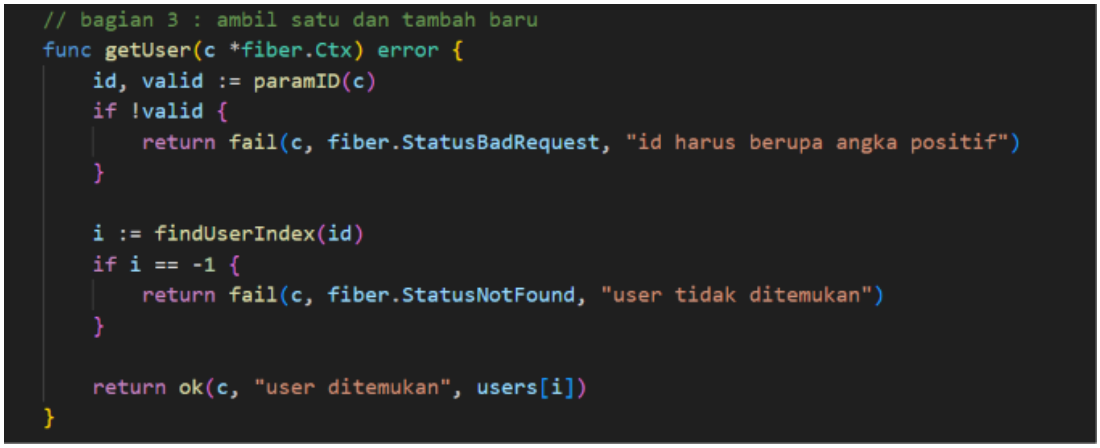
    // 1) Saring
    hasil := []User{}
    for _, u := range users {
        if q.IsActive != nil && u.IsActive != *q.IsActive {
            continue
        }
        if q.Search != "" && !cocokPencarian(u, q.Search) {
            continue
        }
        hasil = append(hasil, u)
    }

    // 2) Urutkan
    sort.SliceStable(hasil, func(i, j int) bool {
        var lebihKecil bool
        switch q.Sort {
        case "username":
            lebihKecil = hasil[i].Username < hasil[j].Username
        case "email":
            lebihKecil = hasil[i].Email < hasil[j].Email
        case "created_at":
            lebihKecil = hasil[i].CreatedAt.Before(hasil[j].CreatedAt)
        default:
            lebihKecil = hasil[i].ID < hasil[j].ID
        }
        if q.Order == "desc" {
            return !lebihKecil
        }
        return lebihKecil
    })
```

```go
// 3) Potong sesuai halaman
total := len(hasil)
totalPages := (total + q.Limit - 1) / q.Limit
mulai := (q.Page - 1) * q.Limit
if mulai > total {
    mulai = total
}
akhir := mulai + q.Limit
if akhir > total {
    akhir = total
}

return okList(c, "daftar user berhasil diambil", hasil[mulai:akhir], &Meta{
    Page: q.Page, Limit: q.Limit, Total: total, TotalPages: totalPages,
})
}
```

Function ini digunakan untuk mengambil daftar user dengan fitur penyaringan, pengurutan, dan pagination. Function menerima c bertipe *fiber.Ctx yang diberikan oleh Fiber ketika request masuk. Dari c, parameter query seperti page, limit, search, sort, order, dan is_active diambil melalui parseListQuery(c). Setelah itu data user dari slice users disaring berdasarkan parameter tersebut, kemudian diurutkan, dipotong sesuai halaman, dan dikirim sebagai response menggunakan okList().



```go
// bagian 3 : ambil satu dan tambah baru
func getUser(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findUserIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "user tidak ditemukan")
    }

    return ok(c, "user ditemukan", users[i])
}
```

Function ini digunakan untuk mengambil satu user berdasarkan ID dari URL. Function menerima c yang diberikan oleh Fiber. ID diambil dari URL melalui paramID(c), kemudian ID tersebut diberikan ke findUserIndex(id) untuk mencari posisi user di dalam slice users. Jika ditemukan, users[i] mengambil data user dari slice users berdasarkan index yang ditemukan, kemudian dikirim sebagai response menggunakan ok(). Jika ID tidak ditemukan, function mengembalikan status 404 Not Found.



```go
func createUser(c *fiber.Ctx) error {
    var req CreateUserRequest
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    errs := map[string]string{}
    req.Username = strings.TrimSpace(req.Username)
    req.Email = strings.TrimSpace(req.Email)

    if req.Username == "" {
        errs["username"] = "wajib diisi"
    }
    if !strings.Contains(req.Email, "@") {
        errs["email"] = "format email tidak valid"
    }
    if len(req.Password) < 8 {
        errs["password"] = "minimal 8 karakter"
    }
    for _, u := range users {
        if strings.EqualFold(u.Username, req.Username) {
            errs["username"] = "sudah dipakai"
        }
    }
    if len(errs) > 0 {
        return failValidation(c, errs)
    }

    baru := User{
        ID:        nextID,
        Username:  req.Username,
        Email:     req.Email,
        Password:  req.Password,
        IsActive:  true,
        CreatedAt: time.Now(),
    }
    users = append(users, baru)
    nextID++

    return created(c, "user berhasil dibuat", baru,
    "/api/v1/users/"+strconv.Itoa(baru.ID))
}
```

Function ini digunakan untuk membuat user baru. Function menerima c yang diberikan oleh Fiber. Data user diperoleh dari JSON request body menggunakan c.BodyParser(&req), sehingga nilai seperti username, email, dan password masuk ke req. Setelah itu data divalidasi. Jika valid, dibuat data User baru. Username, Email, dan Password berasal dari req yang sebelumnya berasal dari JSON body, ID berasal dari variabel program nextID, IsActive ditentukan langsung oleh program dengan nilai true, dan CreatedAt dibuat menggunakan time.Now(). User kemudian ditambahkan ke slice users menggunakan append() dan response 201 Created dikirim.

```go
// bagian 4
// PUT mengganti SELURUH isi. Field yang tidak dikirim dianggap dikosongkan,
// karena itu semuanya wajib ada.
func replaceUser(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findUserIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "user tidak ditemukan")
    }

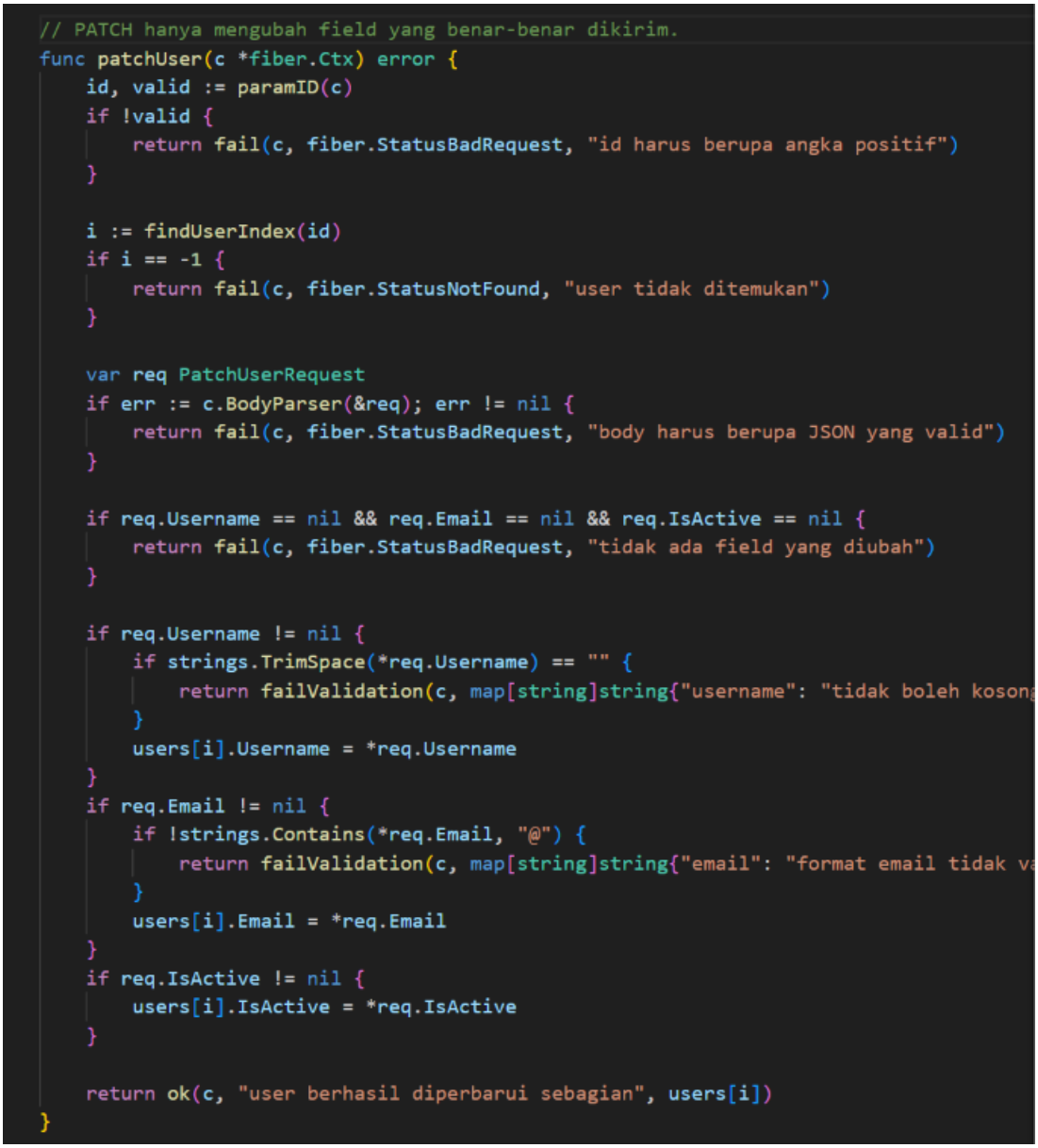
    var req ReplaceUserRequest
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    errs := map[string]string{}
    if strings.TrimSpace(req.Username) == "" {
        errs["username"] = "wajib diisi pada PUT"
    }
    if !strings.Contains(req.Email, "@") {
        errs["email"] = "wajib diisi dan berformat email pada PUT"
    }
    if len(errs) > 0 {
        return failValidation(c, errs)
    }

    users[i].Username = req.Username
    users[i].Email = req.Email
    users[i].IsActive = req.IsActive

    return ok(c, "user berhasil diganti seluruhnya", users[i])
}
```

Function ini digunakan untuk mengganti data user menggunakan metode PUT. Function menerima c yang diberikan oleh Fiber. ID user diperoleh dari URL path melalui paramID(c), sedangkan data penggantinya diperoleh dari JSON request body melalui c.BodyParser(&req). Setelah data divalidasi, username, email, dan status user pada users[i] diganti menggunakan nilai dari req. Jadi pada function ini, ID untuk menentukan user berasal dari URL, sedangkan data yang digunakan untuk mengganti user berasal dari JSON body



```go
// PATCH hanya mengubah field yang benar-benar dikirim.
func patchUser(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findUserIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "user tidak ditemukan")
    }

    var req PatchUserRequest
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    if req.Username == nil && req.Email == nil && req.IsActive == nil {
        return fail(c, fiber.StatusBadRequest, "tidak ada field yang diubah")
    }

    if req.Username != nil {
        if strings.TrimSpace(*req.Username) == "" {
            return failValidation(c, map[string]string{"username": "tidak boleh koson
        }
        users[i].Username = *req.Username
    }
    if req.Email != nil {
        if !strings.Contains(*req.Email, "@") {
            return failValidation(c, map[string]string{"email": "format email tidak v
        }
        users[i].Email = *req.Email
    }
    if req.IsActive != nil {
        users[i].IsActive = *req.IsActive
    }

    return ok(c, "user berhasil diperbarui sebagian", users[i])
}
```

Function ini digunakan untuk mengubah sebagian data user menggunakan metode PATCH. Function menerima c yang diberikan oleh Fiber. ID user diperoleh dari URL path melalui paramID(c), sedangkan data perubahan diperoleh dari JSON request body melalui c.BodyParser(&req). Function kemudian memeriksa field mana yang dikirim. Hanya field yang dikirim yang diperbarui pada users[i], sedangkan field yang tidak dikirim tetap menggunakan nilai sebelumnya.

```go
func deleteUser(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findUserIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "user tidak ditemukan")
    }

    users = append(users[:i], users[i+1:]...)

    return noContent(c) // 204: berhasil, dan memang tidak ada yang perlu dikirim
}
```

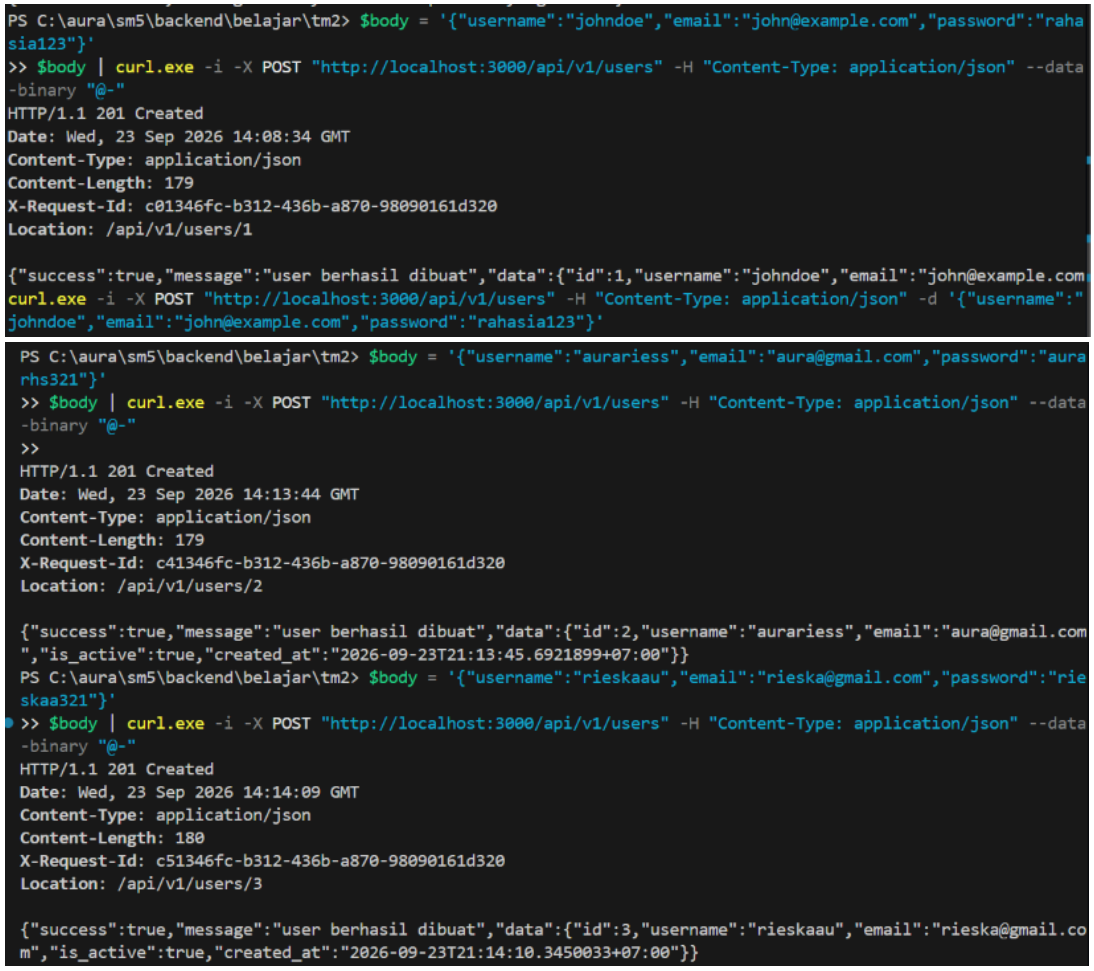
Function ini digunakan untuk menghapus user berdasarkan ID dari URL. Function menerima c yang diberikan oleh Fiber. ID diperoleh dari URL path melalui paramID(c), kemudian diberikan ke findUserIndex(id) untuk mencari index user di dalam slice users. Jika ditemukan, user pada index tersebut dihapus menggunakan append(users[:i], users[i+1:]...). Setelah berhasil dihapus, function mengembalikan status 204 No Content.

## 8. Menguji dan Mengamati

a. Buat User

```
$body =
'{"username":"johndoe","email":"john@example.com","password":"rahasi
a123"}'
$body | curl.exe -i -X POST "http://localhost:3000/api/v1/users" -H
"Content-Type: application/json" --data-binary "@-"
```

Body harus lewat pipe line kalau dari power shell saya.



b. Paginasi dan Pengurutan

```
curl.exe -i
"http://localhost:3000/api/v1/users?page=1&limit=2&sort=username&ord
er=desc"
```

page, limit, dllnya dimasukin ke url.

```
PS C:\aura\sm5\backend\belajar\tm2> curl.exe -i "http://localhost:3000/api/v1/users?page=1&limit=2&sort=userna
me&order=desc"
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 14:15:36 GMT
Content-Type: application/json
Content-Length: 367
X-Request-Id: c61346fc-b312-436b-a870-98090161d320

{"success":true,"message":"daftar user berhasil diambil","data":[{"id":3,"username":"rieskaau","emaususername"
:"rieskaau","email":"rieska@gmail.com","is_active":true,username":"rieskaau","email":"rieska@gmail.com","is_ac
tive":true,"created_at":"2026-09-23T21:14:10.3450033+07:00"},{"id":1,"username":"johndoe","email":"john@userna
username":"rieskaau","email":"rieska@gmail.com","is_active":true,"created_at":"2026-09-23T21:14:10.3450033+07:
00"},{"id":1,"username":"johndoe","email":"john@example.com","is_active":true,"created_at":"2026-09-23T21:08:3
5.5949886+07:00"}],"meta":{"page":1,"limit":2,"total":3,"total_pages":2}}
```

c. Pencarian dan Penyaringan

```
intinya klo curl.exe -i "http://localhost:3000/api/v1/users" semua,
trs klo mau tambahin
kriteria lain pake ? cari based on keyword --> search, kolom=value
untuk cari based on kriteria tertentu
```

```
curl.exe -i
"http://localhost:3000/api/v1/users?search=an&is_active=true"
```

```
PS C:\aura\sm5\backend\belajar\tm2> curl.exe -i "localhost:3000/api/v1/users?search=an&is_active=true"
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 15:15:07 GMT
Content-Type: application/json
Content-Length: 122
X-Request-Id: c71346fc-b312-436b-a870-98090161d320

{"success":true,"message":"daftar user berhasil diambil","data":[],"meta":{"page":1,"limit":10,"total":0,"tota
l_pages":0}}
```

d. PUT – seluruh field wajib

```
$body =
'{"username":"john_baru","email":"jb@example.com","is_active":false}
'
$body | curl.exe -i -X PUT "http://localhost:3000/api/v1/users/1" -H
"Content-Type: application/json" --data-binary "@-"
```

```
PS C:\aura\sm5\backend\belajar\tm2> curl.exe -i "localhost:3000/api/v1/users?search=an&is_active=true"
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 15:15:07 GMT
Content-Type: application/json
Content-Length: 122
X-Request-Id: c71346fc-b312-436b-a870-98090161d320
                                $body = '{"username":"john_baru","email":"jb@example.com","is_active":fals
e}'aftar user berhasil diambil","data":[],"meta":{"page":1,"
>> $body | curl.exe -i -X PUT "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --dat
a-binary "@-"
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 15:17:03 GMT
Content-Type: application/json
Content-Length: 192
X-Request-Id: c81346fc-b312-436b-a870-98090161d320

{"success":true,"message":"user berhasil diganti seluruhnya","data":{"id":1,"username":"john_baru","email":"jb
@example.com","is_active":false,"created_at":"2026-09-23T21:08:35.5949886+07:00"}}
```

e. PUT tanpa email → overwrite isi semua kolom, jika ada kolom yg valuenya tidak diisi pada input maka akan diisi otomatis dengan zero value setiap tipe data atau diisi default.

```
5. PUT tanpa email --> harus pake email
$body = '{"kolom1":"value1","kolomdst":"valuedst"}'
>> $body | curl.exe -i -X PUT "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --data-binary "@-"

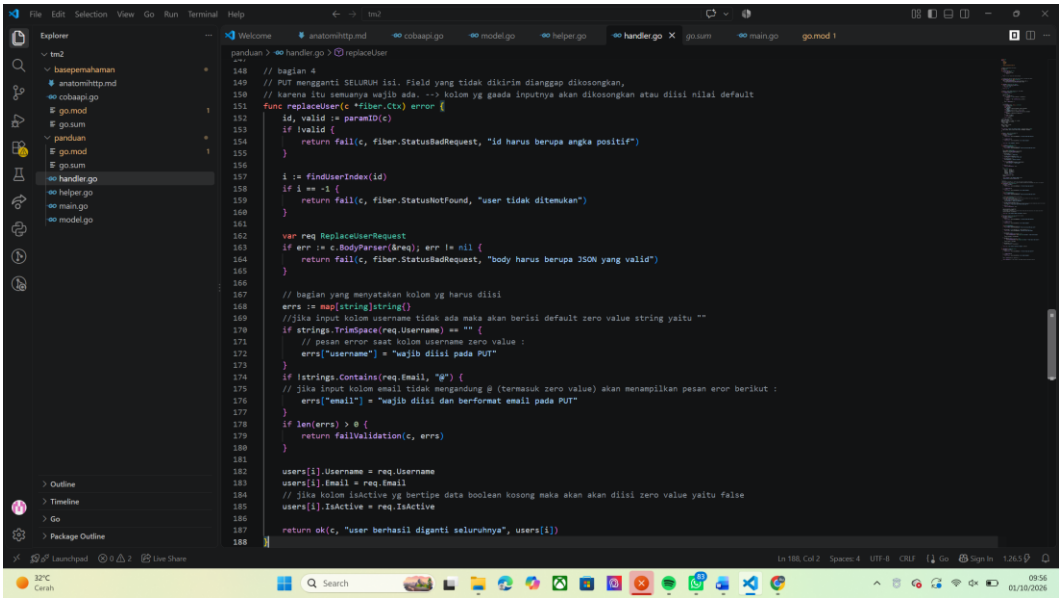
$body = '{"username":"john_biru","email":"jb@example.com"}'
>> $body | curl.exe -i -X PUT "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --data-binary "@-"
```

```
PS C:\aura\sm5\backend\belajar\tm2> $body = '{"username":"john_biru"}'
>> $body | curl.exe -X PUT localhost:3000/api/v1/users/1 -H "Content-Type: application/json" --data-binary "@-
"
>>
{"success":false,"message":"validasi gagal","errors":{"email":"wajib diisi dan berformat email pada PUT"}}
PS C:\aura\sm5\backend\belajar\tm2> $body = '{"username":"john_biru","email":"jb@example.com"}'
>> $body | curl.exe -i -X PUT "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --dat
a-binary "@-"
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 15:29:19 GMT
Content-Type: application/json
Content-Length: 192
X-Request-Id: cc1346fc-b312-436b-a870-98090161d320

{"success":true,"message":"user berhasil diganti seluruhnya","data":{"id":1,"username":"john_biru","email":"jb
@example.com","is_active":false,"created_at":"2026-09-23T21:08:35.5949886+07:00"}}
```

Tidak bisa karena email mandatory.

Kode :



f. PATCH – Sebagian (hanya overwrite kolom yg diminta dalam terminal

```
6. PATCH sebagaian
$body = '{"kolom1":value1}'
$body | curl.exe -X PATCH "http://localhost:3000/api/v1/namaGroupApi/parameter" -H "Content-Type: application/json" --data-binary "@-"

$body = '{"email":"johnpusing@gmail.com"}'
$body | curl.exe -X PATCH "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --data-binary "@-"
```

```
PS C:\aura\sm5\backend\belajar\tm2> $body = '{"email":"johnpusing@gmail.com"}'
>> $body | curl.exe -X PATCH "http://localhost:3000/api/v1/users/1" -H "Content-Type: application/json" --da
ta-binary "@-"
{"success":true,"message":"user berhasil diperbarui sebagian","data":{"id":1,"username":"john_baru","email":
"johnpusing@gmail.com","is_active":false,"created_at":"2026-10-01T10:01:03.7669503+07:00"}}
```

g. Tanpa content-type

Untuk method POST, PUT, PATCH harus pake content-type

```
PS C:\aura\sm5\backend\belajar\tm2> $body = '{"email":"johnpusing@gmail.com"}'
>> $body | curl.exe -X PATCH "http://localhost:3000/api/v1/users/1" --data-binary "@-"
{"success":false,"message":"Content-Type harus application/json"}
PS C:\aura\sm5\backend\belajar\tm2> curl.exe -i "http://localhost:3000/api/v1/users"
HTTP/1.1 200 OK
Date: Thu, 01 Oct 2026 03:43:50 GMT
Content-Type: application/json
Content-Length: 370
X-Request-Id: 6c3a982a-3ce9-4823-92bc-aaf4d930575c

{"success":true,"message":"daftar user berhasil diambil","data":[{"id":1,"username":"john_baru","email":"joh
npusing@gmail.com","is_active":false,"created_at":"2026-10-01T10:01:03.7669503+07:00"},{"id":2,"username":"a
ura","email":"rieska@gmail.com","is_active":true,"created_at":"2026-10-01T10:05:26.391425+07:00"}],"meta":{"
page":1,"limit":10,"total":2,"total_pages":1}}
```

Code :

```go
func main() {

    // requireJSON dipasang khusus pada grup ini, bukan global
    u := api.Group("/users", requireJSON) // require json dipasang untuk semua turunan users
    u.Get("/", listUsers)
    u.Get("/:id", getUser)
    u.Post("/", createUser)
    u.Put("/:id", replaceUser)
    u.Patch("/:id", patchUser)
    u.Delete("/:id", deleteUser)
```

```go
// requireJSON menolak request berisi body yang Content-Type-nya bukan JSON.
// Status yang tepat untuk kasus ini adalah 415, bukan 400.
func requireJSON(c *fiber.Ctx) error {
    if metodeBerbody[c.Method()] {
        ct := c.Get("Content-Type")
        if !strings.HasPrefix(ct, fiber.MIMEApplicationJSON) {
            return fail(c, fiber.StatusUnsupportedMediaType,
                "Content-Type harus application/json")
        }
    }
    return c.Next()
}
```

```go
var metodeBerbody = map[string]bool{
    fiber.MethodPost:  true,
    fiber.MethodPut:   true,
    fiber.MethodPatch: true,
}
```

POST, PUT, dan PATCH wajib Content-Type: application/json karena ketiganya mengirim data lewat body, dan body tersebut akan diparsing oleh BodyParser menjadi struct Go. Content-Type memberi tahu server bahwa data body yang dikirim adalah JSON sehingga bisa diproses dengan benar.

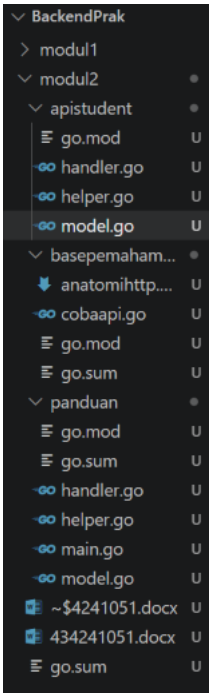
h. Hapus

Delete gaada pesan eror karena pas bikin function dan apinya dibuat noContent dan noContent function berisi menampilkan status saja.

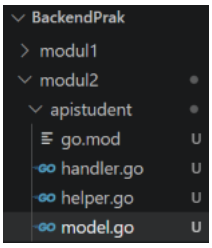


**TUGAS MANDIRI**

1. Entitas dan Struktur Berkas
   a. Buat proyek baru bernama api-students, atau lanjutkan pada folder terpisah dari latihan-fiber ✓




   b. Pecah kode menjadi main.go, model.go, helper.go, dan handler.go ✓




   c. Gunakan struct Student dari tugas pertemuan 1 (ID, Name, Grade, IsActive), lalu tambahkan field NIM sebagai penanda unik

```go
type Student struct{
    //nama di json dan code boleh beda, tp saya mau samain aja biar gak pusing makenya
    ID          int     `json:"id"`
    Name        string  `json:"name"`
    Grade       int     `json:"grade"`
    IsActive    bool    `json:"isactive"`
    NIM         int     `json:"nim"`
}
```

   Deteksi aturan unik atau gak maupun constraint lain dideklarasi melalui function, model hanya deklarasi nama variabel dan tipe datanya

d. Buat struct request terpisah untuk POST, PUT, dan PATCH sebagaimana dicontohkan pada Langkah 2

```go
type requestPostCreateStudent struct{ // asumsi nim bikin sendiri, yg generate id aja trs nanti dibikin contstraint unique
    Name        string  `json:"name"`
    Grade       int     `json:"grade"`
    IsActive    bool    `json:"isactive"`
    NIM         int     `json:"nim"`
}

type requestPutEditSeluruhStudent struct{ // nim gaboleh diedit, setiap kolom klo kosong akan dioverwrite zero value
    Name        string  `json:"name"`
    Grade       int     `json:"grade"`
    IsActive    bool    `json:"isactive"`
}

type requestPatchEditSebagianStudent struct{ // nim gaboleh diedit
    Name        *string `json:"name"` // status boleh gadiisi dan mempertahankan nilai aslinya.
    Grade       int     `json:"grade"` // harus diisi, klo kosonng akan dioverwrite 9
    IsActive    *bool   `json:"isactive"` //status boleh gadiisi dan mempertahankan nilai aslinya.
}
```

notes perbedaan besar :
PUT vs PATCH :
PUT : mengubah semua variabel, klo kosong (nil) akan tetap dikirim dan dioverwrite dengan zero value
PATCH : mengubah sebagian variabel, klo kosong (nil) tidak dikirim (membiarkan kosong atau kalau ada nilai lama maka mempertahankan nilai lama )

TAPI TAPI TAPI
pada patch type data bisa diawali * (misal : *int)--> fungsinya :

| Struct | Request | Yang diterima struct | Hasil |
|---|---|---|---|
| `Age int` | `{}` | `0` | Age jadi 0 |
| `Age *int` | `{}` | `nil` | Age tetap 20 |
| `Age int` | `{"age": 0}` | `0` | Age jadi 0 |
| `Age *int` | `{"age": 0}` | `0` | Age jadi 0 |

intinya,
tanpa * : kosong dianggap 0 --> dikirim zero value
dengan * : kosong dianggap nil --> tidak dikirim

hmm sebenernya yg bikin susah bedain put dan patch example codenya,
disitu put dan patch memang beda tapi validasinya membuat mirip cara kerjanya put dan patch saja : put kosong akan dioverwrite dengan zero value dan patch kosong tidak dikirim (membiarkan kosong atau kalau ada nilai lama maka mempertahankan nilai lama )
validasi :))))  : put "" --> menolak kirim, tidak dikirim jadi yaa sama aja "" cara kerjanya kaya nil di patch yaitu tidak dikirim. dan patch nil dikasih pesan peringartan.
SOOOOOOOO, mending bedain lewat validasi aja drpd pusing bedain patch dan put, lebih customable sesuai kemauan.

e. Terapkan amplop respons yang sama untuk seluruh endpoint, termasuk yang gagal
Di model.go

```go
/*
NO. 1e
buat amplop response
*/
type Meta struct {
    Page       int `json:"page"`
    Limit      int `json:"limit"`
    Total      int `json:"total"`
    TotalPages int `json:"total_pages"`
}

type WebResponse struct {
    Success bool   `json:"success"`
    Message string `json:"message"`
    Data    any    `json:"data,omitempty"`
    Meta    *Meta  `json:"meta,omitempty"`
    Errors  any    `json:"errors,omitempty"`
}
```

1. Deklarasi struct untuk informasi ttg data yg dikirim = meta
2. Deklarasi struct untuk struktur response json

2. Endpoint dan Metode HTTP

```go
func main() {
    api := app.Group("/api/v1") //membuat seluruh endpoint diawal /api/vw

    api.Get("/health", func(c *fiber.Ctx) error {
        return ok(c, "server berjalan", fiber.Map{"timestamp": time.Now()}) //endpoint tambahan untuk cek server jalan atau gak
    })

    // requireJSON dipasang khusus pada grup ini, bukan global
    s := api.Group("/students", requireJSON) // require json dipasang untuk semua turunan student

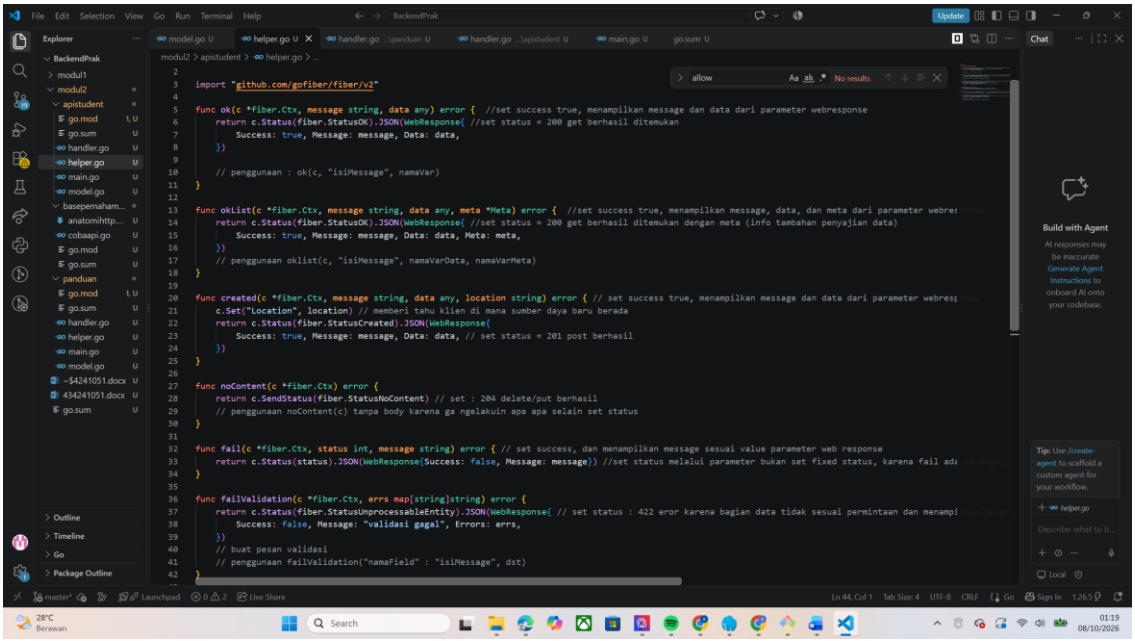
    s.Get("/", getAllStudents)
    s.Get("/:id", getStudent)
    s.Post("/", postCreatetStudent)
    s.Put("/:id", putEditSeluruh)
    s.Patch("/:id", patchEditSebagian)
    s.Delete("/:id", deleteStudent)

    // Endpoint yang tidak dikenal
    app.Use(func(c *fiber.Ctx) error {
        return fail(c, fiber.StatusNotFound, "endpoint tidak ditemukan")
    })

    fmt.Println("Server berjalan di http://localhost:3000")
    log.Fatal(app.Listen(":3000"))
}
```

Implementasi endpoint diawali dengan api := app.Group("/api/v1") yang berfungsi membuat grup dengan prefiks /api/v1, sehingga setiap route yang dibuat melalui api secara otomatis memiliki awalan tersebut. Selanjutnya, s := api.Group("/students", requireJSON) membuat grup khusus untuk resource mahasiswa dengan tambahan prefiks /students, sehingga endpoint di dalam s secara otomatis menjadi /api/v1/students, sekaligus memasang middleware requireJSON yang akan dijalankan terlebih dahulu pada request yang masuk ke seluruh endpoint mahasiswa untuk memastikan request menggunakan format JSON yang sesuai. Setelah grup dibuat, seluruh operasi mahasiswa didaftarkan melalui s, yaitu s.Get("/", getAllStudents), s.Get("/:id", getStudent), s.Post("/", postCreatetStudent), s.Put("/:id", putEditSeluruh), s.Patch("/:id", patchEditSebagian), dan s.Delete("/:id", deleteStudent). Keenam route tersebut menghubungkan metode HTTP dan URL dengan fungsi handler masing-masing, sehingga ketika client mengirim request, Fiber akan mencocokkan metode dan

URL tersebut, menjalankan middleware requireJSON, kemudian meneruskan request ke handler yang sesuai.



```go
import "github.com/gofiber/fiber/v2"

func ok(c *fiber.Ctx, message string, data any) error {  //set success true, menampilkan message dan data dari parameter webresponse
    return c.Status(fiber.StatusOK).JSON(WebResponse{ //set status = 200 get berhasil ditemukan
        Success: true, Message: message, Data: data,
    })

    // penggunaan : ok(c, "isiMessage", namaVar)
}

func oklist(c *fiber.Ctx, message string, data any, meta *Meta) error {  //set success true, menampilkan message, data, dan meta dari parameter webres
    return c.Status(fiber.StatusOK).JSON(WebResponse{ //set status = 200 get berhasil ditemukan dengan meta (info tambahan penyajian data)
        Success: true, Message: message, Data: data, Meta: meta,
    })
    // penggunaan oklist(c, "isiMessage", namaVarData, namaVarMeta)
}

func created(c *fiber.Ctx, message string, data any, location string) error { // set success true, menampilkan message dan data dari parameter webres
    c.Set("Location", location) // memberi tahu klien di mana sumber daya baru berada
    return c.Status(fiber.StatusCreated).JSON(WebResponse{
        Success: true, Message: message, Data: data, // set status = 201 post berhasil
    })
}

func noContent(c *fiber.Ctx) error {
    return c.SendStatus(fiber.StatusNoContent) // set : 204 delete/put berhasil
    // penggunaan noContent(c) tanpa body karena ga ngelakuin apa apa selain set status
}

func fail(c *fiber.Ctx, status int, message string) error { // set success, dan menampilkan message sesuai value parameter web response
    return c.Status(status).JSON(WebResponse{Success: false, Message: message}) //set status melalui parameter bukan set fixed status, karena fail ad
}

func failValidation(c *fiber.Ctx, errs map[string]string) error {
    return c.Status(fiber.StatusUnprocessableEntity).JSON(WebResponse{ // set status : 422 eror karena bagian data tidak sesuai permintaan dan menampi
        Success: false, Message: "validasi gagal", Errors: errs,
    })
    // buat pesan validasi
    // penggunaan failValidation("namaField" : "isiMessage", dst)
}
```

Deklarasikan fungsi-fungsi untuk set status di helper.go (bebas dimana aja cuma biar rapih dan khusus fungsi yg fokus set status)

| Fungsi | Set Status | Kondisi | Return |
|---|---|---|---|
| ok() | 200 OK | Request berhasil dan ingin mengembalikan 1 data/hasil | success, message, data |
| okList() | 200 OK | Request berhasil dan mengembalikan daftar data, terutama yang memakai pagination | success, message, data, meta |
| created() | 201 Created | Request POST berhasil membuat data baru | success, message, data + header Location |
| noContent() | 204 No Content | Operasi berhasil tetapi tidak perlu mengirim body response | Tidak ada body, hanya status 204 |
| fail() | sesuai parameter | Terjadi error yang bukan khusus validasi field, misalnya 400, 404, 415, dll. | success: false, message |
| failValidation() | 422 Unprocessable Entity | Data request masuk, tetapi isi field tidak memenuhi aturan validasi | success: false, "validasi gagal", errors |

```go
// requireJSON menolak request berisi body yang Content-Type-nya bukan JSON.
// Status yang tepat untuk kasus ini adalah 415, bukan 400.
func requireJSON(c *fiber.Ctx) error {
    if metodeBerbody[c.Method()] {
        ct := c.Get("Content-Type")
        if !strings.HasPrefix(ct, fiber.MIMEApplicationJSON) {
            return fail(c, fiber.StatusUnsupportedMediaType,
                "Content-Type harus application/json")
        }
    }
    return c.Next()
}
```

memastikan request yang memiliki body menggunakan Content-Type: application/json. Fungsi terlebih dahulu mengecek method request melalui metodeBerbody. Jika method yang digunakan adalah POST, PUT, atau PATCH, fungsi mengambil nilai Content-Type dari header request. Nilai tersebut kemudian dibandingkan dengan application/json. Jika Content-Type sudah sesuai, fungsi menjalankan c.Next() sehingga request dilanjutkan ke handler endpoint. Jika Content-Type tidak sesuai, fungsi menghentikan proses request dan mengembalikan status 415 Unsupported Media Type karena jenis data yang dikirim tidak sesuai dengan format yang dibutuhkan API.

```go
func findStudentIndex(id int) int {
    for i := range students {
        if students[i].ID == id {
            return i
        }
    }
    return -1 //eror untuk data tidak ditemukan
}
```

mencari posisi data mahasiswa berdasarkan ID. Fungsi melakukan perulangan terhadap seluruh data pada slice students, kemudian membandingkan students[i].ID dengan ID yang diterima sebagai parameter. Jika ditemukan ID yang sama, fungsi mengembalikan index data mahasiswa tersebut agar handler dapat mengambil atau mengubah data pada posisi tersebut. Jika seluruh data sudah diperiksa tetapi tidak ada ID yang sesuai, fungsi mengembalikan -1 sebagai tanda bahwa mahasiswa tidak ditemukan. Nilai -1 tersebut kemudian digunakan oleh handler untuk menghasilkan status 404 Not Found.

```go
// mendeteksi NIM ganda
func findStudentByNIM(nim int) int {
    for i := range students {
        if students[i].NIM == nim {
            return i
        }
    }
    return -1
}
```

mengecek apakah NIM yang dimasukkan sudah digunakan oleh mahasiswa lain. Fungsi bekerja dengan melakukan perulangan pada seluruh data mahasiswa yang tersimpan di students, kemudian membandingkan nilai nim dari setiap mahasiswa dengan nilai nim yang diterima sebagai parameter. Jika ditemukan mahasiswa dengan NIM yang sama, fungsi mengembalikan index mahasiswa tersebut. Jika seluruh data sudah diperiksa tetapi tidak ditemukan NIM yang sama, fungsi mengembalikan -1 yang menandakan bahwa NIM belum digunakan.

```go
func paramID(c *fiber.Ctx) (int, bool) {
    id, err := strconv.Atoi(c.Params("id"))
    if err != nil || id < 1 {
        return 0, false
    }
    return id, true
}
```

mengambil dan memvalidasi nilai id yang terdapat pada URL. Nilai id dari c.Params("id") awalnya berbentuk string sehingga fungsi menggunakan strconv.Atoi() untuk mengubahnya menjadi integer. Setelah dikonversi, fungsi juga memastikan ID lebih besar dari 0. Jika proses konversi gagal, misalnya ID berisi "abc", atau hasil konversi kurang dari 1, fungsi mengembalikan false sehingga handler menghentikan proses dan mengembalikan status 400 Bad Request. Jika ID valid, fungsi mengembalikan nilai ID beserta true agar dapat digunakan untuk mencari data mahasiswa.

```go
func getAllStudents(c *fiber.Ctx) error {
    // mengambil seluruh data mahasiswa
    data := students

    // search berdasarkan nama, tidak membedakan huruf besar dan kecil
    search := strings.TrimSpace(c.Query("search"))
    if search != "" {
        var filtered []Student

        for _, student := range data {
            if strings.Contains(
                strings.ToLower(student.Name),
                strings.ToLower(search),
            ) {
                filtered = append(filtered, student)
            }
        }

        data = filtered
    }
```

Fungsi getAllStudents() mengambil page dan limit dari query string dengan nilai default masing-masing 1 dan 10. Setelah memastikan keduanya positif dan page tidak melebihi jumlah halaman, fungsi menentukan data berdasarkan posisi start dan end. Data kemudian dikirim bersama informasi pagination melalui okList() dengan status 200 OK.

```go
func getStudent(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findStudentIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "mahasiswa tidak ditemukan")
    }

    return ok(c, "mahasiswa ditemukan", students[i])
}
```

Fungsi getStudent() mengambil dan memvalidasi ID menggunakan paramID(), kemudian mencari mahasiswa menggunakan findStudentIndex(). Jika ID tidak valid, dikembalikan 400 Bad Request. Jika mahasiswa tidak ditemukan, dikembalikan 404 Not Found. Jika ditemukan, data mahasiswa dikirim dengan 200 OK.



```go
func postCreatetStudent(c *fiber.Ctx) error {
    var req requestPostCreateStudent
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    errs := map[string]string{}
    req.Name = strings.TrimSpace(req.Name)

    if req.Name == "" {
        errs["name"] = "wajib diisi"
    }
    if req.Grade < 0 || req.Grade > 100 {
        errs["grade"] = "harus antara 0 sampai 100"
    }
    if req.NIM < 1 {
        errs["nim"] = "wajib diisi dan berupa angka positif"
    }
    if len(errs) > 0 {
        return failValidation(c, errs)
    }

    // NIM ganda = bertentangan dengan data yang ada (409), dicek setelah validasi
    if findStudentByNIM(req.NIM) != -1 {
        return fail(c, fiber.StatusConflict, "NIM sudah terdaftar")
    }

    baru := Student{
        ID:       nextID,
        Name:     req.Name,
        Grade:    req.Grade,
        IsActive: req.IsActive,
        NIM:      req.NIM,
    }
    students = append(students, baru)
    nextID++

    return created(c, "mahasiswa berhasil ditambahkan", baru,
        "/api/v1/students/"+strconv.Itoa(baru.ID))
}
```

Fungsi postCreatetStudent() membaca body JSON menggunakan BodyParser(), kemudian memvalidasi name, grade, dan nim. Jika validasi gagal, dikembalikan 422 Unprocessable Entity. Setelah validasi berhasil, fungsi mengecek NIM menggunakan findStudentByNIM(). Jika NIM sudah digunakan, dikembalikan 409 Conflict. Jika belum, data mahasiswa baru dibuat, dimasukkan ke students, nextID dinaikkan, kemudian dikembalikan dengan 201 Created dan header Location.

```go
func putEditSeluruh(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findStudentIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "mahasiswa tidak ditemukan")
    }

    var req requestPutEditSeluruhStudent
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    // bagian yang menyatakan kolom yg harus diisi
    errs := map[string]string{}
    //jika input kolom name tidak ada maka akan berisi default zero value string yai
    if strings.TrimSpace(req.Name) == "" {
        // pesan error saat kolom name zero value :
        errs["name"] = "wajib diisi pada PUT"
    }
    if req.Grade < 0 || req.Grade > 100 {
        errs["grade"] = "harus antara 0 sampai 100"
    }
    if len(errs) > 0 {
        return failValidation(c, errs)
    }

    students[i].Name = strings.TrimSpace(req.Name)
    students[i].Grade = req.Grade
    // jika kolom isActive yg bertipe data boolean kosong maka akan akan diisi zero
    students[i].IsActive = req.IsActive

    return ok(c, "mahasiswa berhasil diganti seluruhnya", students[i])
}
```

Fungsi putEditSeluruh() mengambil ID dari URL dan mencari mahasiswa yang akan diubah. Setelah body JSON berhasil dibaca, name dan grade divalidasi. Jika validasi gagal, dikembalikan 422 Unprocessable Entity. Jika berhasil, nilai Name, Grade, dan IsActive pada data lama diganti dengan nilai dari request, kemudian dikembalikan dengan 200 OK.

```go
func patchEditSebagian(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findStudentIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "mahasiswa tidak ditemukan")
    }

    var req requestPatchEditSebagianStudent
    if err := c.BodyParser(&req); err != nil {
        return fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    // Grade bertipe int biasa, jadi dianggap tidak dikirim kalau 0
    if req.Name == nil && req.IsActive == nil && req.Grade == 0 {
        return fail(c, fiber.StatusBadRequest, "tidak ada field yang diubah")
    }

    if req.Name != nil {
        if strings.TrimSpace(*req.Name) == "" {
            return failValidation(c, map[string]string{"name": "tidak boleh kosong"}
        }
        students[i].Name = strings.TrimSpace(*req.Name)
    }
    if req.Grade < 0 || req.Grade > 100 {
        return failValidation(c, map[string]string{"grade": "harus antara 0 sampai 1
    }
    students[i].Grade = req.Grade // harus diisi, kalau kosong dioverwrite 0 (sesuai
    if req.IsActive != nil {
        students[i].IsActive = *req.IsActive
    }

    return ok(c, "mahasiswa berhasil diperbarui sebagian", students[i])
}
```

Fungsi patchEditSebagian() mencari mahasiswa berdasarkan ID, kemudian membaca body JSON. Field Name dan IsActive menggunakan pointer sehingga dapat dibedakan antara field yang dikirim dan tidak dikirim. Field yang dikirim kemudian diperbarui, sedangkan field yang tidak dikirim tetap dipertahankan. Jika tidak ada field yang diubah, dikembalikan 400 Bad Request. Hasil perubahan dikembalikan dengan 200 OK.

```go
func deleteStudent(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    i := findStudentIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "mahasiswa tidak ditemukan")
    }

    students = append(students[:i], students[i+1:]...)

    return noContent(c) // 204: berhasil, dan memang tidak ada yang perlu dikirim
}
```

Fungsi deleteStudent() mengambil ID, mencari mahasiswa menggunakan findStudentIndex(), lalu menghapus data tersebut dari slice students menggunakan append(students[:i], students[i+1:]...). Jika berhasil dihapus, fungsi mengembalikan 204 No Content tanpa body.

3. **Query String**
   a. Page dan limit

```go
func getAllStudents(c *fiber.Ctx) error {
    // mengambil page dan limit dari query string
    // default page = 1, limit = 10
    page := c.QueryInt("page", 1)
    limit := c.QueryInt("limit", 10)

    // page dan limit harus positif
    if page < 1 {
        return fail(c, fiber.StatusBadRequest, "page harus berupa angka positif")
    }

    if limit < 1 {
        return fail(c, fiber.StatusBadRequest, "limit harus berupa angka positif")
    }

    // batas maksimal limit = 100
    // agar client tidak meminta terlalu banyak data dalam satu request
    if limit > 100 {
        return fail(c, fiber.StatusBadRequest, "limit maksimal 100")
    }
```

Parameter page digunakan untuk menentukan halaman data yang ingin ditampilkan, sedangkan limit menentukan jumlah maksimal mahasiswa dalam satu halaman. Jika parameter tidak dikirim, page menggunakan nilai bawaan 1 dan limit menggunakan nilai bawaan 10. Setelah diambil, keduanya diperiksa agar bernilai positif. Selain itu, limit dibatasi maksimal 100 untuk mencegah client meminta data dalam jumlah terlalu besar dalam satu request sehingga dapat meningkatkan beban server. Setelah parameter valid, nilai tersebut digunakan untuk menentukan posisi awal dan akhir data yang akan ditampilkan.

   b. Search nama

```go
    // search berdasarkan nama, tidak membedakan huruf besar dan kecil
    search := strings.TrimSpace(c.Query("search"))
    if search != "" {
        var filtered []Student

        for _, student := range data {
            if strings.Contains(
                strings.ToLower(student.Name),
                strings.ToLower(search),
            ) {
                filtered = append(filtered, student)
            }
        }

        data = filtered
    }
```

Parameter search digunakan untuk mencari mahasiswa berdasarkan nama. Jika search dikirim, program melakukan perulangan pada data mahasiswa dan membandingkan nama setiap mahasiswa dengan kata yang dicari. Sebelum dibandingkan, nama mahasiswa dan nilai search diubah menjadi huruf kecil

menggunakan strings.ToLower(), sehingga pencarian tidak membedakan huruf besar dan kecil. Jika nama mahasiswa mengandung kata yang dicari, data tersebut dimasukkan ke hasil filtered.

c. Sort dan order

```
// mengambil parameter sort dan order
sortField := c.Query("sort", "id")
order := strings.ToLower(c.Query("order", "asc"))

// whitelist field yang boleh digunakan untuk sorting
allowedSort := map[string]bool{
    "id":        true,
    "name":      true,
    "grade":     true,
    "is_active": true,
    "nim":       true,
}

if !allowedSort[sortField] {
    return fail(c, fiber.StatusBadRequest,
        "field sort tidak diizinkan")
}

if order != "asc" && order != "desc" {
    return fail(c, fiber.StatusBadRequest,
        "order harus asc atau desc")
}
```

Parameter sort digunakan untuk menentukan field yang menjadi dasar pengurutan, sedangkan order menentukan arah pengurutan. Jika tidak diberikan, data secara default diurutkan berdasarkan id dengan urutan asc. Field yang dapat digunakan untuk sorting dibatasi menggunakan allowedSort sebagai whitelist, yaitu id, name, grade, is_active, dan nim. Pembatasan ini memastikan client hanya dapat melakukan sorting pada field yang telah ditentukan. Sementara itu, order hanya menerima asc atau desc. Jika nilai sort atau order tidak sesuai ketentuan, request ditolak dengan status 400 Bad Request.

```
// sorting berdasarkan field yang sudah masuk whitelist
sort.Slice(data, func(i, j int) bool {
    var hasil bool

    switch sortField {
    case "id":
        hasil = data[i].ID < data[j].ID
    case "name":
        hasil = strings.ToLower(data[i].Name) < strings.ToLower(data[j].Name)
    case "grade":
        hasil = data[i].Grade < data[j].Grade
    case "is_active":
        hasil = !data[i].IsActive && data[j].IsActive
    case "nim":
        hasil = data[i].NIM < data[j].NIM
    }

    if order == "desc" {
        return !hasil
    }

    return hasil
})
```

Setelah field dan arah pengurutan dinyatakan valid, sort.Slice() digunakan untuk mengurutkan data. Program menentukan field yang digunakan melalui switch, kemudian membandingkan nilai antar mahasiswa. Jika order bernilai asc, data diurutkan dari nilai lebih kecil ke lebih besar atau secara alfabetis untuk nama. Jika order bernilai desc, urutan dibalik menjadi dari besar ke kecil.

d. Filter

```
// filter berdasarkan is_active
isActive := c.Query("is_active")
if isActive != "" {
    var filtered []Student

    for _, student := range data {
        if isActive == "true" && student.IsActive {
            filtered = append(filtered, student)
        } else if isActive == "false" && !student.IsActive {
            filtered = append(filtered, student)
        }
    }

    if isActive != "true" && isActive != "false" {
        return fail(c, fiber.StatusBadRequest,
            "is_active harus true atau false")
    }

    data = filtered
}
```

Filter yang digunakan adalah is_active berdasarkan nilai status aktif mahasiswa. Jika parameter bernilai true, program hanya memasukkan mahasiswa dengan IsActive bernilai true ke hasil. Jika bernilai false, hanya mahasiswa yang tidak

aktif yang dimasukkan. Nilai parameter dibatasi hanya true atau false; jika client mengirim nilai lain, request ditolak dengan status 400 Bad Request.

e. Meta

```
// jumlah data setelah search dan filter
total := len(data)

// menghitung jumlah halaman
totalPages := (total + limit - 1) / limit

if totalPages == 0 {
    totalPages = 1
}

if page > totalPages {
    return fail(c, fiber.StatusBadRequest,
        "page melebihi jumlah halaman")
}

// menentukan data sesuai page dan limit
start := (page - 1) * limit
end := start + limit

if end > total {
    end = total
}

data = data[start:end]

// informasi pagination
meta := &Meta{
    Page:       page,
    Limit:      limit,
    Total:      total,
    TotalPages: totalPages,
}
```

Setelah proses search dan filter, total dihitung berdasarkan jumlah data yang tersisa. Nilai tersebut digunakan untuk menghitung jumlah halaman berdasarkan limit. Informasi pagination kemudian disimpan dalam Meta, yang terdiri dari page, limit, total, dan total_pages. Dengan demikian, client tidak hanya mendapatkan daftar mahasiswa, tetapi juga mengetahui posisi halaman dan jumlah keseluruhan data yang sesuai dengan query yang digunakan.

Contoh request :
GET
/api/v1/students?page=1&limit=10&search=andi&sort=grade&order=desc&is_active=true
Request tersebut akan mengambil mahasiswa aktif yang namanya mengandung andi, mengurutkannya berdasarkan grade dari terbesar ke terkecil, dan menampilkan maksimal 10 data pada halaman pertama.

4. **Status Code dan Header**

a. 200 Pengambilan atau perubahan berhasil

```
func ok(c *fiber.Ctx, message string, data any) error {  //set success true, menampi
    return c.Status(fiber.StatusOK).JSON(WebResponse{ //set status = 200 get berhasi
        Success: true, Message: message, Data: data,
    })

    // penggunaan : ok(c, "isiMessage", namaVar)
}
```

Fungsi ok() digunakan ketika request berhasil diproses dan server perlu mengembalikan data kepada client. Fungsi menetapkan status 200 OK, kemudian response dikemas dalam WebResponse yang berisi Success, Message, dan Data. Fungsi ini digunakan pada endpoint seperti GET satu mahasiswa, PUT, dan PATCH ketika proses berhasil.

```
func okList(c *fiber.Ctx, message string, data any, meta *Meta) error {  //set succe
    return c.Status(fiber.StatusOK).JSON(WebResponse{ //set status = 200 get berhasi
        Success: true, Message: message, Data: data, Meta: meta,
    })
    // penggunaan oklist(c, "isiMessage", namaVarData, namaVarMeta)
}
```

Fungsi okList() digunakan ketika endpoint mengembalikan kumpulan data mahasiswa. Selain data, response juga membutuhkan informasi pagination sehingga Meta ikut dikirim. Fungsi ini menetapkan status 200 OK dan digunakan oleh endpoint GET daftar mahasiswa.

b. 201 Penambahan berhasil, disertai header Location

```
func created(c *fiber.Ctx, message string, data any, location string) error { // set
    c.Set("Location", location) // memberi tahu klien di mana sumber daya baru berad
    return c.Status(fiber.StatusCreated).JSON(WebResponse{
        Success: true, Message: message, Data: data, // set status = 201 post berhas
    })
}
```

Fungsi created() digunakan setelah proses POST berhasil membuat mahasiswa baru. Server menetapkan status 201 Created untuk menunjukkan bahwa resource baru berhasil dibuat. Selain itu, header Location diatur menggunakan c.Set() dan berisi URL resource yang baru dibuat. Pada postCreatetStudent(), nilai tersebut dibentuk berdasarkan ID mahasiswa baru sehingga client dapat mengetahui lokasi resource tersebut.

c. 204 Penghapusan berhasil

```
func noContent(c *fiber.Ctx) error {
    return c.SendStatus(fiber.StatusNoContent) // set : 204 delete/put berhasil
    // penggunaan noContent(c) tanpa body karena ga ngelakuin apa apa selain set sta
}
```

Fungsi noContent() digunakan setelah data mahasiswa berhasil dihapus. Server mengembalikan status 204 No Content, yang menunjukkan bahwa request berhasil

tetapi tidak ada data yang perlu dikirim dalam response body. Fungsi ini dipanggil oleh deleteStudent() setelah data berhasil dihapus dari students.

```
func fail(c *fiber.Ctx, status int, message string) error { // set success, dan mena
    return c.Status(status).JSON(WebResponse{Success: false, Message: message}) //se
}
```

Fungsi fail() digunakan untuk mengirim response error dengan status code yang ditentukan oleh handler. Nilai status dikirim sebagai parameter status, misalnya fiber.StatusBadRequest untuk 400 atau fiber.StatusNotFound untuk 404. Nilai tersebut kemudian digunakan oleh c.Status(status) untuk menetapkan HTTP status pada response. Dengan demikian, satu fungsi fail() dapat digunakan untuk berbagai jenis error sesuai kondisi yang terjadi pada handler.

d. 400 Body bukan JSON yang sah

```
func getStudent(c *fiber.Ctx) error {
    id, valid := paramID(c)
    if !valid {
        return fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }
```

Status 400 Bad Request digunakan ketika request yang diterima tidak memenuhi format atau parameter yang dibutuhkan, seperti ID yang bukan angka positif, nilai page atau limit yang tidak valid, serta body yang bukan JSON valid. Status ditetapkan dengan mengirim fiber.StatusBadRequest sebagai parameter status ke fungsi fail(), kemudian nilai tersebut digunakan oleh c.Status(status) untuk menghasilkan response HTTP 400

e. 404 Data Tidak Ditemukan

```
func getStudent(c *fiber.Ctx) error {
    i := findStudentIndex(id)
    if i == -1 {
        return fail(c, fiber.StatusNotFound, "mahasiswa tidak ditemukan")
    }

    return ok(c, "mahasiswa ditemukan", students[i])
}
```

Status 404 Not Found digunakan ketika resource atau endpoint yang diminta tidak ditemukan. Pada pencarian mahasiswa, status ini diberikan ketika findStudentIndex() mengembalikan -1, yang berarti tidak ada mahasiswa dengan ID yang diminta. Status 404 juga digunakan pada middleware terakhir di main.go ketika request tidak cocok dengan route yang tersedia. Nilai fiber.StatusNotFound kemudian diteruskan ke fungsi fail() dan digunakan oleh c.Status(status) untuk menetapkan response HTTP 404.

f. 409 Bertentangan dengan data yang ada

```
func postCreatetStudent(c *fiber.Ctx) error {

    // NIM ganda = bertentangan dengan data yang ada (409), dicek setelah validasi lolos
    if findStudentByNIM(req.NIM) != -1 {
        return fail(c, fiber.StatusConflict, "NIM sudah terdaftar")
    }
```

Status 409 Conflict diterapkan pada proses POST ketika NIM yang dikirim sudah terdaftar. Kondisi tersebut diperiksa menggunakan findStudentByNIM(). Jika fungsi menemukan NIM yang sama dan mengembalikan index selain -1, handler memanggil fail() dengan fiber.StatusConflict. Nilai tersebut kemudian digunakan oleh c.Status(status) untuk menetapkan HTTP status 409 Conflict pada response.

g. 415 Content-Type bukan aplicaton/json

```
// requireJSON menolak request berisi body yang Content-Type-nya bukan JSON.
// Status yang tepat untuk kasus ini adalah 415, bukan 400.
func requireJSON(c *fiber.Ctx) error {
    if metodeBerbody[c.Method()] {
        ct := c.Get("Content-Type")
        if !strings.HasPrefix(ct, fiber.MIMEApplicationJSON) {
            return fail(c, fiber.StatusUnsupportedMediaType,
                "Content-Type harus application/json")
        }
    }
    return c.Next()
}
```

Status 415 Unsupported Media Type digunakan ketika request POST, PUT, atau PATCH memiliki Content-Type yang bukan application/json. Pengecekan dilakukan oleh middleware requireJSON() dengan mengambil nilai Content-Type dari request dan membandingkannya dengan application/json. Jika tidak sesuai, middleware memanggil fail() dengan fiber.StatusUnsupportedMediaType, sehingga response menghasilkan status HTTP 415. Jika sesuai, request diteruskan ke proses berikutnya menggunakan c.Next().

h. 422 Validasi isi gagal, rincian per field

```
func failValidation(c *fiber.Ctx, errs map[string]string) error {
    return c.Status(fiber.StatusUnprocessableEntity).JSON(WebResponse{ // set status : 422 eror karena bagian data
        Success: false, Message: "validasi gagal", Errors: errs,
    })
    // buat pesan validasi
    // penggunaan failValidation("namaField" : "isiMessage", dst)
}
```

Status 422 Unprocessable Entity digunakan ketika request memiliki format yang dapat diproses, tetapi nilai data yang dikirim tidak memenuhi aturan validasi. Pada kode ini, ketika validasi pada proses POST, PUT, atau PATCH menemukan kesalahan, handler memanggil fungsi failValidation(). Fungsi tersebut menggunakan fiber.StatusUnprocessableEntity melalui c.Status() sehingga response menghasilkan status HTTP 422 dan mengirimkan detail kesalahan pada bagian Errors.